37.3 远程访问

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~210k tokens，输出 ~6k tokens（本节）

本地部署的 Gateway 默认绑定在 `127.0.0.1`（loopback），仅允许同一台机器上的客户端连接。但实际场景中，用户经常需要从笔记本、手机或远程服务器访问家中或办公室的 Gateway。OpenClaw 提供了四种远程访问方案：**Tailscale Serve/Funnel**、**SSH 隧道**、**服务发现（mDNS/Bonjour）** 和**广域 DNS-SD**。这些方案可以组合使用——例如，用 Tailscale 做安全传输层，同时用 Bonjour 做局域网自动发现。

37.3.1 Tailscale Serve / Funnel

Tailscale 简介

> **衍生解释**：Tailscale 是基于 WireGuard 协议的零配置 VPN。它为每台设备分配一个 100.64.0.0/10 网段的 IPv4 地址和一个 `<hostname>.<tailnet>.ts.net` 形式的 DNS 名称。设备之间的流量经过端到端加密，无需手动配置防火墙或端口转发。**Tailscale Serve** 允许将本地端口暴露给同一 tailnet 内的其他设备；**Tailscale Funnel** 则进一步将端口暴露到公网（需要在管理控制台启用权限）。

三种模式

OpenClaw Gateway 的 Tailscale 集成通过配置项 `gateway.tailscale.mode` 控制，支持三种模式：

模式	可访问范围	认证方式	典型场景
`off`	仅本地	token/password	默认，不启用 Tailscale
`serve`	tailnet 内部	Tailscale 身份 + token	家庭/办公室内多设备
`funnel`	公网	password（强制）	公开 webhook、外部协作

源码实现

Tailscale 的核心逻辑位于 `src/gateway/server-tailscale.ts` （59 行）和 `src/infra/tailscale.ts` （496 行）。Gateway 启动时调用 `startGatewayTailscaleExposure()` ：

```
// src/gateway/server-tailscale.ts（简化）
export async function startGatewayTailscaleExposure(params: {
  tailscaleMode: "off" | "serve" | "funnel";
  resetOnExit?: boolean;
  port: number;
  controlUiBasePath?: string;
}) {
  if (params.tailscaleMode === "off") return null;

  // 根据模式调用不同的 Tailscale CLI 命令
  if (params.tailscaleMode === "serve") {
    await enableTailscaleServe(params.port);    // tailscale serve --bg --yes <port>
  } else {
    await enableTailscaleFunnel(params.port);   // tailscale funnel --bg --yes <port>
  }

  // 获取 tailnet 主机名用于日志
  const host = await getTailnetHostname().catch(() => null);
  // 输出: "serve enabled: https://myhost.tail12345.ts.net/ (WS via wss://...)"

  // 返回清理函数（进程退出时调用）
  if (!params.resetOnExit) return null;
  return async () => {
    if (params.tailscaleMode === "serve") {
      await disableTailscaleServe();    // tailscale serve reset
    } else {
      await disableTailscaleFunnel();   // tailscale funnel reset
    }
  };
}
```

底层的 `enableTailscaleServe()` / `enableTailscaleFunnel()` 实际上是执行系统上的 Tailscale CLI：

```
// src/infra/tailscale.ts（简化）
export async function enableTailscaleServe(port: number) {
  const bin = await getTailscaleBinary();
  await execWithSudoFallback(bin, ["serve", "--bg", "--yes", String(port)], {
    maxBuffer: 200_000,
    timeoutMs: 15_000,
  });
}
```

Tailscale 二进制定位

`findTailscaleBinary()` 使用四级回退策略定位 `tailscale` 可执行文件：

1. **PATH 查找**： `which tailscale`
2. **macOS 已知路径**： `/Applications/Tailscale.app/Contents/MacOS/Tailscale`
3. **find 搜索**：在 `/Applications` 下搜索 `Tailscale.app`
4. **locate 数据库**：使用 `locate Tailscale.app` 作为最后手段

找到后结果会被缓存，避免重复搜索。

权限提升与 sudo 回退

某些 Tailscale 操作（如 `serve` 、 `funnel` ）可能需要 root 权限。OpenClaw 实现了自动 sudo 回退机制：

```
async function execWithSudoFallback(bin, args, opts) {
  try {
    return await exec(bin, args, opts);           // 先尝试直接执行
  } catch (err) {
    if (!isPermissionDeniedError(err)) throw err;
    return await exec("sudo", ["-n", bin, ...args], opts);  // 用 sudo -n 重试
  }
}
```

`-n` （non-interactive）标志确保 sudo 不会阻塞等待密码输入——如果用户没有配置免密 sudo，则直接失败并抛出原始错误。

Tailscale Whois 认证

当 Gateway 运行在 Serve 模式时，Tailscale 的反向代理会注入身份头：

HTTP 头	内容
`tailscale-user-login`	用户的 Tailscale 登录名
`tailscale-user-name`	用户的显示名称
`tailscale-user-profile-pic`	头像 URL

Gateway 通过 `readTailscaleWhoisIdentity()` 验证这些头——它调用 `tailscale whois --json <ip>` 查询连接者的真实 Tailscale 身份，然后与 HTTP 头中的 login 进行交叉比对。验证结果使用 LRU 缓存（成功 TTL 60 秒，失败 TTL 5 秒），避免对每次请求都执行系统调用。

安全约束

运行时配置检查（`server-runtime-config.ts`）强制执行两条安全规则：

1. **Funnel 要求密码认证**：`funnel` 模式下必须设置 `gateway.auth.mode = "password"`，因为 Funnel 暴露到公网，仅靠 token 不够安全。
2. **Serve/Funnel 要求 loopback 绑定**：Tailscale 代理通过 `127.0.0.1` 转发流量，Gateway 不应同时监听外部接口。

```
if (tailscaleMode === "funnel" && authMode !== "password") {
  throw new Error("tailscale funnel requires gateway auth mode=password");
}
if (tailscaleMode !== "off" && !isLoopbackHost(bindHost)) {
  throw new Error("tailscale serve/funnel requires gateway bind=loopback");
}
```

37.3.2 SSH 隧道

SSH 隧道是最通用的远程访问方式——不依赖任何第三方服务，只要远程机器开放了 SSH 端口即可。

隧道建立流程

OpenClaw 的 SSH 隧道实现位于 `src/infra/ssh-tunnel.ts`（214 行），核心函数 `startSshPortForward()` 完成以下步骤：

```
┌──────────────┐        SSH -L 转发        ┌──────────────┐
│  本地客户端   │ ──127.0.0.1:本地端口── │  远程 Gateway  │
│  (CLI/macOS) │                          │ 127.0.0.1:18789│
└──────────────┘                          └──────────────┘
```

1. **解析目标**：`parseSshTarget()` 支持多种格式：`user@host`、`user@host:port`、`host:port`、甚至带 `ssh` 前缀。默认端口 22。
2. **端口分配**：优先使用首选端口（通常是 18789），若被占用则分配临时端口。
3. **启动 SSH 进程**：使用 `spawn("/usr/bin/ssh", args)` 创建隧道。
4. **等待就绪**：轮询 `127.0.0.1:本地端口` 直到可以建立 TCP 连接。

SSH 参数详解

`startSshPortForward()` 构造的 SSH 命令包含精心选择的参数：

```
const args = [
  "-N",                                  // 不执行远程命令
  "-L", `${localPort}:127.0.0.1:${remotePort}`,  // 本地端口转发
  "-p", String(parsed.port),             // 远程 SSH 端口
  "-o", "ExitOnForwardFailure=yes",      // 转发失败时立即退出
  "-o", "BatchMode=yes",                 // 禁止交互式密码提示
  "-o", "StrictHostKeyChecking=accept-new",  // 首次连接自动接受主机密钥
  "-o", "UpdateHostKeys=yes",            // 自动更新已知主机密钥
  "-o", "ConnectTimeout=5",              // 连接超时 5 秒
  "-o", "ServerAliveInterval=15",        // 每 15 秒发送心跳
  "-o", "ServerAliveCountMax=3",         // 3 次心跳无响应则断开
  "--", userHost,                        // '--' 防止主机名被解析为选项
];
```

> **衍生解释**：`-L localPort:host:remotePort` 是 SSH 本地端口转发（Local Port Forwarding）的标准语法。它在本地机器上监听 `localPort`，将收到的数据通过加密 SSH 通道转发到远程机器上的 `host:remotePort`。由于 Gateway 只监听 `127.0.0.1`，这里的 host 也是 `127.0.0.1`，即远程机器的 loopback。

安全防护

SSH 隧道实现包含两个重要的安全措施：

1. **主机名注入防御**：`parseSshTarget()` 拒绝以 `-` 开头的主机名，防止恶意输入被解释为 SSH 命令行选项。

2. **`--` 参数终止符**：在主机名前插入 `--`，确保即使主机名包含特殊字符也不会被误解析。

生命周期管理

隧道进程的停止函数实现了优雅关闭：先发送 `SIGTERM`，等待最多 1.5 秒，若进程仍未退出则发送 `SIGKILL` 强制终止。

```
const stop = async () => {
  child.kill("SIGTERM");
  await new Promise<void>((resolve) => {
    const t = setTimeout(() => {
      child.kill("SIGKILL");
      resolve();
    }, 1500);
    child.once("exit", () => { clearTimeout(t); resolve(); });
  });
};
```

配置集成

用户可以在 `~/.openclaw/config.yaml` 中配置远程 Gateway 的 SSH 连接参数：

```
gateway:
  mode: remote
  remote:
    url: ws://127.0.0.1:18789
    transport: ssh           # 使用 SSH 隧道
    sshTarget: user@myserver.local
    sshIdentity: ~/.ssh/id_ed25519
    token: <gateway-token>
```

CLI 也支持通过命令行参数 `--ssh <target>` 直接建立隧道。

37.3.3 服务发现 — mDNS/Bonjour

当 Gateway 和客户端在同一局域网时，用户不应该需要手动输入 IP 地址和端口。OpenClaw 使用 **mDNS/Bonjour**（也称为 DNS-SD，DNS-based Service Discovery）实现零配置服务发现。

> **衍生解释**：mDNS（Multicast DNS）是一种在局域网内实现 DNS 解析的协议（RFC 6762），无需中心 DNS 服务器。设备通过向 224.0.0.251:5353 发送组播查询来发现其他设备。DNS-SD（DNS-

> based Service Discovery，RFC 6763）在 mDNS 之上定义了服务注册和发现的标准格式。Apple 的 Bonjour 是 mDNS + DNS-SD 的实现。

广播端——Bonjour Advertiser

Gateway 启动时通过 `startGatewayBonjourAdvertiser()` （ `src/infra/bonjour.ts` ）向局域网广播自己的存在。底层使用 `@homebridge/ciao` 库——这是 Homebridge 项目维护的纯 JavaScript mDNS 实现。

服务注册

```
const gateway = responder.createService({
  name: safeServiceName(instanceName),  // 如 "myhost (OpenClaw)"
  type: "openclaw-gw",                  // 服务类型
  protocol: Protocol.TCP,
  port: opts.gatewayPort,               // 如 18789
  domain: "local",
  hostname,                             // 如 "myhost"
  txt: gatewayTxt,                      // TXT 记录
});
```

注册的 DNS-SD 服务类型为 `_openclaw-gw._tcp` ， TXT 记录包含了客户端连接所需的元数据：

TXT 键	内容	minimal 模式
`role`	`gateway`	✓
`gatewayPort`	端口号	✓
`lanHost`	`hostname.local`	✓
`displayName`	人类可读名称	✓
`transport`	`gateway`	✓
`gatewayTls`	`1` （如已启用 TLS）	✓
`tailnetDns`	Tailscale DNS 名称	✓
`sshPort`	SSH 端口	✗（仅 full 模式）
`cliPath`	CLI 路径	✗（仅 full 模式）

三种广播模式

mDNS 广播通过 `discovery.mdns.mode` 配置：

- **`minimal`** （默认）：省略 `sshPort` 和 `cliPath` ，减少信息暴露
- **`full`** ：完整广播所有元数据
- **`off`** ：完全禁用 mDNS

Watchdog 机制

网络接口变化（如 WiFi 重连、睡眠唤醒）可能导致 mDNS 广播失效。Bonjour Advertiser 内置了一个 60 秒间隔的 watchdog，它检测服务状态是否仍为 `announced` ，若不是则尝试重新广播：

```
const watchdog = setInterval(() => {
  for (const { label, svc } of services) {
    if (svc.serviceState === "announced" || svc.serviceState === "announcing") continue
    // 状态异常，尝试重新广播
    svc.advertise().catch(/* ... */);
  }
}, 60_000);
watchdog.unref();  // 不阻止进程退出
```

发现端——Beacon Discovery

客户端通过 `discoverGatewayBeacons()` （ `src/infra/bonjour-discovery.ts` ，604 行）发现 Gateway。该函数根据平台选择不同的发现工具：

平台	工具	命令
macOS	`dns-sd` （系统内置）	`dns-sd -B _openclaw-gw._tcp local.`
Linux	`avahi-browse`	`avahi-browse -rt _openclaw-gw._tcp`

发现流程：

1. **浏览**：先列出指定域内所有 `_openclaw-gw._tcp` 实例
2. **解析**：对每个发现的实例查询其完整信息（主机、端口、TXT 记录）
3. **解码**：处理 DNS-SD 特有的八进制转义（如中文名称）

返回的 `GatewayBonjourBeacon` 对象包含了客户端连接所需的全部信息，macOS 应用和 iOS 应用的配对流程就是基于此实现的（参见第 28 章）。

37.3.4 广域 DNS-SD（Wide-Area Discovery）

标准 mDNS 只在局域网内有效——组播包不会穿越路由器。OpenClaw 支持**广域 DNS-SD**（也称 Unicast DNS-SD），通过常规 DNS 记录（非组播）实现跨网络的服务发现。

工作原理

广域 DNS-SD 的核心思想是将 mDNS 记录写入一个真实的 DNS 域：

```
; 域名: openclaw.internal.
_openclaw-gw._tcp.openclaw.internal.  IN PTR  myhost-gateway._openclaw-gw._tcp.openclaw
myhost-gateway._openclaw-gw._tcp.openclaw.internal.  IN SRV  0 0 18789 myhost.openclaw.
myhost-gateway._openclaw-gw._tcp.openclaw.internal.  IN TXT  "displayName=My Host" "rol
myhost.openclaw.internal.  IN A   100.64.1.42
```

Zone 文件生成

`startGatewayDiscovery()`（`src/gateway/server-discovery-runtime.ts`）在启动时检查是否启用了广域发现：

```
if (params.wideAreaDiscoveryEnabled) {
  const wideAreaDomain = resolveWideAreaDiscoveryDomain(/* ... */);
  const tailnetIPv4 = pickPrimaryTailnetIPv4();
  await writeWideAreaGatewayZone({
    domain: wideAreaDomain,
    gatewayPort: params.port,
    displayName: formatBonjourInstanceName(params.machineDisplayName),
    tailnetIPv4,
    tailnetIPv6: pickPrimaryTailnetIPv6() ?? undefined,
    // ...
  });
}
```

`writeWideAreaGatewayZone()`（`src/infra/widearea-dns.ts`，200 行）生成标准的 BIND 格式 zone 文件，并使用**内容哈希**（FNV-1a）实现幂等写入——只有记录实际发生变化时才更新文件和递增 SOA serial。

发现端的广域回退

客户端发现时，如果标准 mDNS 没有在广域域（如 `openclaw.internal.` ）发现实例，会触发 `discoverWideAreaViaTailnetDns()` 回退：

1. 通过 `tailscale status --json` 获取 tailnet 中所有节点的 IPv4 地址
2. 以 6 路并发向每个 IP 发送 `dig PTR` 查询
3. 找到能响应的节点后，继续查询 SRV 和 TXT 记录
4. 组装成与 mDNS 相同格式的 `GatewayBonjourBeacon`

这种方式结合了 Tailscale 的设备列表和 DNS-SD 的标准协议，无需搭建中心 DNS 服务器即可实现跨网络发现。

配置示例

```yaml
discovery:
  mdns:
    mode: minimal          # 或 full / off
  wideArea:
    enabled: true
    domain: openclaw.internal
```

对应的环境变量：

环境变量	作用
`OPENCLAW_DISABLE_BONJOUR=1`	完全禁用 Bonjour
`OPENCLAW_MDNS_HOSTNAME`	自定义 mDNS 主机名
`OPENCLAW_WIDE_AREA_DOMAIN`	广域发现域名
`OPENCLAW_TAILNET_DNS`	手动指定 Tailscale DNS 名称
`OPENCLAW_SSH_PORT`	mDNS TXT 中的 SSH 端口（full 模式）
`OPENCLAW_CLI_PATH`	手动指定 CLI 路径

本节小结

1. **Tailscale Serve/Funnel** 提供零配置的安全远程访问。Serve 模式限 tailnet 内部，Funnel 暴露到公网（强制密码认证 + loopback 绑定）。底层通过调用 Tailscale CLI 实现，支持 sudo 回退和 Whois 身份验证。
2. **SSH 隧道** 是最通用的方案。 `startSshPortForward()` 自动分配本地端口、设置安全参数（心跳、超时、防注入）、轮询等待隧道就绪，并提供优雅关闭机制。
3. **mDNS/Bonjour** 实现局域网零配置发现。Gateway 通过 `@homebridge/ciao` 广播 `_openclaw-gw._tcp` 服务，客户端通过 `dns-sd` （macOS）或 `avahi-browse` （Linux）发现。Watchdog 确保睡眠唤醒后自动恢复。
4. **广域 DNS-SD** 突破局域网限制，通过生成标准 DNS zone 文件 + Tailscale 节点扫描实现跨网络发现，无需搭建中心化 DNS 基础设施。
5. 这四种方案可灵活组合：Tailscale 做传输层 + Bonjour 做发现层是最常见的搭配。